Chapter 8: Software Testing

1. Program Testing Goals

1. To demonstrate to the developer and the customer that the software meets its requirements.
    - *Custom software:* At least one test for every requirement.
    - *Generic software:* Tests for all features and combinations.
2. To discover situations in which the behavior of the software is incorrect, undesirable, or does not conform to its specification.
    - *Defect testing:* Rooting out system crashes, unwanted interactions, incorrect computations, and data corruption.

2. Software Inspections

- People examine the source representation to discover anomalies and defects.
- Does not require execution of a system, so it can be used before implementation.
- Can be applied to any representation (requirements, design, configuration data, test data).
- **Advantages:**
    - During active testing, errors can mask (hide) other errors. Inspection is static, so you don't have to worry about interactions between errors.
    - Incomplete versions can be inspected without additional costs (no test harnesses needed).
    - Can also consider broader quality attributes (compliance with standards, portability, maintainability).

3. Stages of Testing

1. **Development testing:** System is tested during development to discover bugs and defects.
2. **Release testing:** A separate team tests a complete version before it is released to users.
3. **User testing:** Users or potential users test the system in their own environment.

4. Development Testing (Deep Dive)

Carried out by the developing team. Includes:

- **Unit testing:** Individual components tested in isolation (defect testing). Units can be functions/methods, object classes, or composite components.

    - *Object class testing coverage includes:* Testing all operations, setting/interrogating all attributes, exercising the object in all possible states. *Inheritance makes this difficult as information is not localized.*
- **Component testing:** Several individual units integrated to test component interfaces.
- **System testing:** Some or all components integrated to test component interactions.

Automated Testing:

- Unit testing should be automated to run/check without manual intervention.
- Uses a test automation framework (like JUnit) that provides generic test classes you extend to create specific cases. Reports success/failure often through a GUI.

5. Testing Strategies

- **Partition testing:** Identify groups of inputs with common characteristics that should be processed the same way.
    - Inputs/outputs fall into classes (an **equivalence partition** or domain). Test cases should be chosen from each partition.
- **Guideline-based testing:** Choose test cases based on guidelines reflecting previous experience of common programmer errors.
    - *General Guidelines:* Force the system to generate all error messages, overflow input buffers, repeat the same input numerous times, force invalid outputs, force computation results to be too large or too small.

6. Test-Driven Development (TDD)

- An approach where you inter-leave testing and code development.
- Tests are written before the code; passing the tests is the critical driver.
- You develop incrementally and do not move on until the increment passes its test.
- Introduced as part of agile methods (like Extreme Programming) but usable in plan-driven processes.

TDD Process Activities:

1. Identify increment of functionality (usually small, a few lines of code).
2. Write an automated test for this functionality.
3. Run the test along with all others. The new test will fail (since it's not implemented yet).
4. Implement the functionality and re-run the test.
5. Once successful, move on to the next chunk of functionality.

Benefits of TDD:

- **Code coverage:** Every segment written has at least one associated test.
- **Regression testing:** A regression test suite is developed incrementally.

- **Simplified debugging:** When a test fails, the problem lies in the newly written code.
- **System documentation:** The tests describe what the code should be doing.

7. Release Testing

- Testing a particular release of a system intended for use outside the development team.
- **Primary Goal:** To convince the supplier that the system is good enough for use (delivers specified functionality, performance, dependability, and won't fail during normal use).
- Usually a **black-box testing process** (tests derived only from the system specification).
- *Note on differences from System Testing:* Release testing is a form of system testing, but it is done by a **separate team**. Development system testing focuses on bugs (**defect testing**), while release testing checks requirements (**validation testing**).
- **Features tested by scenario examples:** Authentication, downloading/uploading records, home visit scheduling, encryption/decryption on mobile, record retrieval/modification, database links, call prompting.

8. User Testing

- Users or customers provide input and advice on system testing.
- Essential because the user's working environment has a major effect on reliability, performance, usability, and robustness (which cannot be replicated in a testing environment).

Types of User Testing:

1. **Alpha testing:** Users work with the development team at the developer's site.
2. **Beta testing:** A release is made available to users to experiment with and raise problems discovered.
3. **Acceptance testing:** Customers test a system to decide if it is ready to be accepted from developers and deployed (primarily for custom systems).